DBMON-06: Dashboards and Alerting

> **Series:** DBMON — Database Monitoring | **Notebook:** 6 of 7 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

This notebook covers building database monitoring dashboards and configuring alerting for database health. You will learn dashboard design patterns for database KPIs, how to create alert-ready queries for slow query detection, connection pool thresholds, error rate monitoring, and database-specific SLO definitions. These queries are designed to be directly usable in **Dashboards (new)** tiles and in DQL-based **custom alerts (Davis anomaly detectors)** routed through Workflows.

Table of Contents

1. Database KPIs
2. Health Overview Dashboard
3. Response Time Monitoring
4. Error Rate Alerting
5. Throughput and Capacity
6. Slow Query Alerting
7. Database SLO Definitions
8. Summary

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there)
OneAgent	Deployed on application hosts with database clients
Permissions	`storage:spans:read`, `storage:metrics:read`, `storage:entities:read`
Data	Active database traffic across monitored services
Prior Reading	DBMON-01 through DBMON-05 for context on database monitoring techniques

1. Database KPIs

Effective database monitoring revolves around a small set of key performance indicators. Each KPI maps to a specific dashboard tile and potential alert condition.

The alert thresholds in this notebook are community starting points — Dynatrace publishes none of them. Tune each against your own baseline.

KPI	What It Measures	Dashboard Tile	Alert Threshold
Response Time (P95)	Query latency at the 95th percentile	Timeseries chart	> 500ms sustained
Error Rate	Percentage of failed database calls	Single value + trend	> 1% of total calls
Throughput	Queries per second	Timeseries chart	Sudden drop > 50%
Slow Query Count	Queries exceeding threshold	Counter + list	> 10 per 5min window
Connection Errors	Timeout and refused connections	Counter	Any occurrence

KPI	What It Measures	Dashboard Tile	Alert Threshold
Top Queries by Time	Highest-impact query patterns	Table	Total time > SLO budget

2. Health Overview Dashboard

The health overview provides a single-pane summary of all database systems. Use these queries as dashboard tiles for an executive-level view.

```
// Dashboard tile: Database health summary — one row per database system
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    total_calls = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    error_count = countIf(span.status_code == "error"),
    slow_count = countIf(duration &gt; 500ms),
    unique_databases = countDistinct(db.namespace)
}, by:{db.system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
toDouble(total_calls)) * 100, decimals:2)
| fieldsAdd slow_rate_pct = round((toDouble(slow_count) /
toDouble(total_calls)) * 100, decimals:2)
| sort total_calls desc
```

```
// Dashboard tile: Total database calls — single value
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize total_db_calls = count()
```

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Dashboard tile: Database calls per service — identify heaviest consumers
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
}, by:{dt.service.name}
| sort call_count desc
| limit 10
```

Health Score as a Prioritization Signal

The queries above build a custom health overview from spans. If the **Databases** app is enabled (its docs list vendors *"such as PostgreSQL, MySQL, MSSQL, Oracle, MariaDB, and HanaDB"* — verify it is enabled on your tenant), its instance table shows health per instance — *"Health highlights instances with critical, warning, or no active alerts"* — a faster first pass for "which database needs attention right now" than scanning this dashboard row by row. (Per instance, the health-score concept page documents a weighted 0–100 score: *"The health score is the weighted average of all checks."*)

> **Sources:** Databases app (DT docs) — *"Health highlights instances with critical, warning, or no active alerts"*, Database health score overview (DT docs) — *"The health score is the weighted average of all checks."*

Use the two together rather than choosing one: the Health Score triages *which* instance to look at, and the span-based queries in this section remain the way to build custom dashboard tiles, thresholds, and SLOs the app's UI doesn't expose.

3. Response Time Monitoring

Response time monitoring is the most critical aspect of database dashboards. These queries provide both real-time and trend views.

```
// Dashboard tile: Database response time trend by system (6-hour view)
fetch spans, from:-6h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries p95_ms = percentile(duration / 1ms, 95),
                 by:{db.system},
                 interval:5m
```

```
// Dashboard tile: Response time by database instance (server.address)
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(server.address)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms
}, by:{db.system, server.address, db.namespace}
| sort p95_ms desc
| limit 15
```

```
// Dashboard tile: P50 vs P95 vs P99 comparison — all databases combined
fetch spans, from:-6h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries {
    p50_ms = percentile(duration / 1ms, 50),
    p95_ms = percentile(duration / 1ms, 95),
    p99_ms = percentile(duration / 1ms, 99)
  }, interval:5m
```

4. Error Rate Alerting

Database errors (connection timeouts, deadlocks, constraint violations) should trigger alerts when they exceed normal baseline levels.

```
// Alert query: Database error rate per 5-minute window
fetch spans, from:-1h
| filter isNotNull(db.system)
| makeTimeseries {
    total = count(),
    errors = countIf(span.status_code == "error", default:0)
  }, by:{db.system}, interval:5m
```

```
// Alert query: Error breakdown by type — classify error categories
// span.status_message is experimental and is often empty on failed DB spans;
the error
// text is recorded as an exception event in span.events. expand counts
exception events,
// so a span with two exception events counts twice.
fetch spans, from:-1h
| filter isNotNull(db.system) and span.status_code == "error"
| expand span.events
| filter isNotNull(span.events[exception.type])
| fieldsAdd exception_type = span.events[exception.type], exception_message =
span.events[exception.message]
| summarize error_count = count(), by:{db.system, server.address,
exception_type, exception_message}
| sort error_count desc
| limit 20
```

```
// Alert query: Error rate trend over 24 hours — detect escalating problems
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries {
    total = count(),
    errors = countIf(span.status_code == "error", default:0)
  }, interval:30m
```

Recommended Alert Thresholds

These are starting points from community practice, not Dynatrace guidance.

Condition	Severity	Alert When
Error rate > 5% for 5 minutes	Critical	Immediate page
Error rate > 1% for 15 minutes	Warning	Notification

Condition	Severity	Alert When
Any connection refused error	Critical	Immediate page
Deadlock (engine-side)	Warning	Notification — alert on the **Databases** app's lock and wait metrics; spans see a deadlock only as the victim session's error

A deadlock happens inside the engine, so none of the span queries in this notebook detects one. For monitored database instances, the lock and wait metrics shown in the **Databases** app cover it, and health alerts can be created on them. In a client span the deadlock appears only as the error returned to the session the engine terminated.

> **Sources:** Database blocking queries and deadlocks (DT docs) — *"The engine detects the cycle and terminates one session (the deadlock victim), which must retry."* and *"Create health alerts on lock and wait metrics to be notified when contention reaches a threshold."*

Alerting on Engine Internals — SQL Server Extension Metrics

The alert queries above are span-based (caller-side). If the **Microsoft SQL Server extension** is deployed (see DBMON-02 §6 for the feature sets), the engine's own health signals become alertable too — as **metric events or Davis anomaly detectors** on the `sql-server.*` keys, in the same problem/notification pipeline:

Signal	Metric / stream	Starting point
Transaction log filling up	`sql-server.databases.log.percentUsed`	Metric event at > 80%, warn at > 70%
Engine blocking	`sql-server.general.processesBlocked`	Davis anomaly detector (baseline beats a static count)
Always On degradation	`sql-server.always-on.ag.synchronizationHealth`	Metric event on any drop below healthy
Agent job failures	`failed_jobs` / `current_jobs` **log streams**	DQL log alert — the failure message text is part of the alert context

The **Starting point** column is this series' recommendation, not a published Dynatrace value — tune each threshold against your own baseline before alerting on it. Thresholds carried over from homegrown scripts should likewise be revalidated rather than copied — e.g., scripts typically count *blocking* SPIDs while the extension metric counts *blocked* processes. The decision framework for migrating a script/Telegraf estate onto these signals is in **FAQ-14**.

Log-space alert query — databases whose transaction log crossed 80% in the window:

> **Sources:** Microsoft SQL Server extension (DT docs) — the `sql-server.*` metric keys, the `failed_jobs` / `current_jobs` log streams, and the feature sets that emit them.

```
// Alert query: transaction logs above 80% utilization (SQL Server extension)
timeseries logUsedPct = avg(`sql-server.databases.log.percentUsed`), from:-24h
| fieldsAdd worst = arrayMax(logUsedPct)
| filter worst &gt; 80
```

```
// Alert query: failed SQL Server Agent jobs with failure context (Jobs
feature set)
// The extension writes failed jobs to their own log stream: dt.extension.name
identifies the
// extension and event.group == "failed_jobs" the stream (field names per the
extension docs).
fetch logs, from:-24h
| filter matchesValue(dt.extension.name, "com.dynatrace.extension.sql-server")
| filter matchesValue(event.group, "failed_jobs")
| summarize {failures = count(), latest = max(timestamp)}, by:{job_name,
server}
| sort failures desc
```

5. Throughput and Capacity

Monitoring query throughput helps detect traffic anomalies and plan for capacity. A sudden drop in throughput may indicate an upstream failure, while a spike may indicate a runaway process.

```
// Dashboard tile: Queries per minute by database system
fetch spans, from:-6h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries qpm = count(), by:{db.system}, interval:1m
```

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Dashboard tile: SQL operation mix over time — read vs write trend,
classified from the statement text
// db.operation.name alone misclassifies batched writes (named SET) and counts
driver phases (DBMON-02 § 4).
// For MongoDB, DynamoDB or Redis use the command lists in DBMON-03 § 6 and
DBMON-04 § 2.
fetch spans, from:-6h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"PREPARE", "RESULTSET"}))
| fieldsAdd q = upper(trim(db.query.text))
| fieldsAdd op_type = if(startsWith(q, "SELECT") or startsWith(q, "WITH"),
then:"READ",
    else:if(contains(q, "INSERT ") or contains(q, "UPDATE ") or contains(q,
"DELETE ") or contains(q, "MERGE "), then:"WRITE",
    else:"OTHER"))
| makeTimeseries op_count = count(), by:{op_type}, interval:5m
```

```
// Dashboard tile: Query volume — the last 24 hours vs the 24 hours before
// Rolling windows, not calendar days. Scans 48 h of spans: run it on demand
rather than as an
// auto-refreshing tile.
fetch spans, from:-24h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize count = count()
| fieldsAdd period = "last_24h"
| append [
    fetch spans, from:-48h, to:-24h
    | filter isNotNull(db.system)
    | filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
    | summarize count = count()
    | fieldsAdd period = "previous_24h"
  ]
| fields period, count
```

6. Slow Query Alerting

Slow query alerts detect when query performance degrades beyond acceptable thresholds.

Recommended path: wire these queries into a **custom alert (Davis anomaly detector)** — created in **Settings** from SaaS 1.344, or in the Anomaly Detection app on earlier versions (SaaS 1.344 rolls out to tenants in stages; check your tenant's version, and note the app is deprecated) — and route the resulting events with **Workflows**, so Davis can apply adaptive baselines, multi-dimensional analysis, and seasonal pattern detection — far more accurate than fixed thresholds for slow-query detection. See the **AIOPS** series for anomaly-detection mechanisms and the **WFLOW** series for Workflow-driven alert routing.

Fixed thresholds: a custom alert does not have to be adaptive. Its alert condition also takes a threshold value you set manually, so the same DQL queries cover fixed-threshold cases. Classic **metric events** are not a route for these span queries: they are metric-based, and custom alerts are the DQL path that extends them. An existing metric event can be transformed into a custom alert.

> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*; Configure a simple custom alert (DT docs) — *"To manually create or edit a simple custom alert configuration, you can use Anomaly Detection or Settings."* and *"Because Anomaly Detection is deprecated, we recommend using Settings instead."*, *"You can also choose the desired threshold value and the Unit of your value manually."* and *"Metric events enhance anomaly detection, expanding them beyond out-of-the-box use cases into metric-based events. With the power of DQL, you can extend this reach further."* **Derived:** that metric events do not take `fetch spans` DQL follows from their being metric-based.

```
// Alert query: Slow query count per 5-minute window (&gt; 500ms threshold)
fetch spans, from:-1h
| filter isNotNull(db.system)
| filter duration &gt; 500ms
| makeTimeseries slow_count = count(), by:{db.system}, interval:5m
```

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Alert query: Current slow query detail — for investigation
fetch spans, from:-15m
| filter isNotNull(db.system)
| filter duration &gt; 500ms
| fields start_time, db.system, db.namespace, db.operation.name,
        db.query.text, server.address,
        duration_ms = duration / 1ms,
        dt.service.name
| sort duration_ms desc
| limit 20
```

```
// Alert query: P95 response time exceeding SLO threshold
fetch spans, from:-6h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries p95_ms = percentile(duration / 1ms, 95),
                 by:{db.system, server.address},
                 interval:5m
```

Slow Query Alert Thresholds

Starting points from community practice — set yours from each system's observed baseline.

Database Type	Warning Threshold	Critical Threshold	Window
SQL (PostgreSQL, MySQL, etc.)	P95 > 200ms	P95 > 500ms	5 minutes
NoSQL (MongoDB, DynamoDB)	P95 > 100ms	P95 > 300ms	5 minutes
Cache (Redis, Memcached)	P95 > 5ms	P95 > 20ms	5 minutes
Search (Elasticsearch)	P95 > 500ms	P95 > 2000ms	5 minutes

7. Database SLO Definitions

Service Level Objectives (SLOs) for databases define the expected performance contract. Use these queries to measure SLO compliance.

Recommended Database SLOs

SLO	Target	Measurement
Availability	99.9% of calls succeed	Error rate < 0.1%
Latency	95% of calls under threshold	P95 < vendor-specific threshold
Throughput	No more than 50% drop from baseline	Queries/min vs 7-day average

```
// SLO measurement: Database availability — success rate over 24 hours
// span.status_code is set only on FAILED database spans (value "error"); a
successful call
// leaves it null. `!= "error"` therefore evaluates to null, not true, and
counts nothing —
// so successes are derived by subtraction rather than by a negative
comparison.
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
}, by:{db.system}
| fieldsAdd successful = total - errors
| fieldsAdd availability_pct = round((toDouble(successful) / toDouble(total))
* 100, decimals:3)
| sort availability_pct asc
```

```
// SLO measurement: Latency compliance — percentage of calls under threshold
fetch spans, from:-24h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    total = count(),
    under_threshold = countIf(duration &lt; 500ms)
}, by:{db.system}
| fieldsAdd latency_slo_pct = round((toDouble(under_threshold) /
toDouble(total)) * 100, decimals:2)
| sort latency_slo_pct asc
```

```
// SLO trend: Hourly availability over 24 hours
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries {
    total = count(),
    errors = countIf(span.status_code == "error", default:0)
  }, by:{db.system}, interval:1h
```

8. Summary

In this notebook you learned:

- The key database KPIs to monitor: response time, error rate, throughput, slow queries, and connection health
- Dashboard-ready queries for a unified database health overview
- Response time monitoring with P50/P95/P99 trend analysis
- Error rate alerting queries and recommended thresholds
- Throughput monitoring including read/write ratio trends and day-over-day comparison
- Slow query alerting with vendor-specific thresholds (DQL custom alerts — Davis anomaly detectors — with an adaptive baseline or a fixed threshold)
- Database SLO definitions and compliance measurement queries

Series Complete

This is the final substantive notebook in the DBMON series. DBMON-99 is the consolidated best-practice summary.

For a complete database monitoring implementation:

1. **DBMON-01** — Understand fundamentals and establish baselines
2. **DBMON-02** — Deep dive into SQL database monitoring
3. **DBMON-03** — NoSQL database monitoring patterns
4. **DBMON-04** — Cache and messaging system monitoring
5. **DBMON-05** — Advanced query analysis and optimization
6. **DBMON-06** — (This notebook) Dashboards, alerting, and SLOs
7. **DBMON-99** — Best Practice Summary

Where to Go Deeper

- **AIOPS series** (8 notebooks) — Davis AI: anomaly detection mechanisms (static / auto-adaptive / seasonal / multi-dimensional baseline / novelty/forecast), Davis problems & RCA, Davis CoPilot
- **WFLOW series** (12 notebooks) — Workflow-driven alert routing, AI tasks, MCP server integration
- **DASH series** (8 notebooks) — Dashboard strategy, executive reporting, tile-design patterns
- **FAQ-02** — Tagging strategy for ownership routing in alerts

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.